Higher National Diploma in Software Engineering

Machine Learning Documentation

2025.2F

School of Computing and Engineering

National Institute of Business Management Kandy

Group Members

Name	Index No
D.C.L Robin	KAHNDSE252F-003
R.M Hamdhan	KAHNDSE252F-011

CONTENTS

Acknowledgement

I would like to express my heartfelt appreciation to all lecturers for their ongoing dedication to education and the profound impact they continue to have on students myself. In an era of rapid technological change and evolving educational paradigms, Lecturers stands out as a beacon of adaptability and innovation. Their commitment to staying at the forefront of modules ensures that we, as students, are always receiving the most current and relevant knowledge.

As lecturers continues to inspire and educate, they leave an indelible mark on each student fortunate enough to learn from them. Their ongoing passion for teaching and commitment to excellence serve as a powerful reminder of the transformative power of education.

Thank You, all the lecturers, for your tireless efforts, your innovative spirit, and the lasting impact you continue to make on our educational journey and future careers.

1. Introduction

Sales forecasting is important for supermarket businesses because it supports inventory planning, staff scheduling, promotion planning, and profit improvement. When sales are not predicted properly, supermarkets may face overstocking, understocking, wastage, and poor business decisions.

This project builds a Machine Learning based forecasting system using a supermarket transaction dataset. The main prediction target is gross income. The model learns from historical sales details such as branch, city, customer type, gender, product line, unit price, quantity, date, time, payment method, customer rating, longitude, and latitude.

The project follows the coursework requirement: exploratory data analysis, reusable preprocessing, model selection, model tuning, and practical model deployment through a mobile application.

2. Problem Statement

Competition is stiff in the supermarket business. Thus, it is necessary to have reliable sales forecasting capabilities for good inventory management, knowing how to schedule staff and promote the products. Unreliable forecasting will create the risk of too much fresh food being ordered and not having enough stock of other products in demand.

The purpose of the project is to develop a comprehensive solution for machine learning which will help in forecasting the gross revenue obtained from each of supermarkets. The implementation of the program is designed so that, with the help of the historical data acquired from transactions in supermarkets from January to March 2019, it will be possible to analyze the tendencies of the time, the type of goods and whether the payment is made in cash or plastic card.

The system will operate in the form of a mobile application which will provide the information in real-time through the smartphone of the manager.

3 Project Objectives

The project goals are defined as follows:

- Analyze the supermarket sales data by means of applying statistical summaries and data visualization techniques.
- Carry out the data cleansing process in order to eliminate the missing values of data and bring the dataset into the desired format so that further modeling could be performed.
- Create the variables related to time and sales, such as the month, the day of the week, hour of the day, whether or not it is a weekend, and lag variables and rolling averages of the current values.
- Create and evaluate at least two regression models by using the criterion of RMSE, MAE, and R2.
- Apply the time series validation procedure in order to adjust the model that demonstrates the best performance level in order to prevent its overfitting.

4. Dataset Overview

The selected dataset is the publicly available Supermarket Sales dataset on Kaggle. (Kaggle, 2019) It contains real-world point-of-sale transaction records from a supermarket chain operating across three branches in Myanmar

Attributes	**Details**
Dataset Name	Supermarket Sales
File Format	CSV (.csv)
Number of Rows	1,000 transaction records
Number of columns	17 columns
Target Variable	Target Variable
Problem typr	Regression/ sales forecasting
Time span	January 2019 – March 2019 (3 months)
Main Output	Predicted gross income

Important Data Correction

According to the proposal document, it was mentioned that the dataset must include 17 columns, while the analysis of the uploaded CSV file showed that there were 20 columns with missing values. As a result, the final report and preprocessing section used the actual dataset information.

4.1Key Columns

Column	Type	Purpose
Branch	Categorical	Supermarket branch.
City	Categorical	City where the branch operates.
Customer type	Categorical	Member or normal customer.
Gender	Categorical	Customer gender.
Product line	Categorical	Product category.
Unit price	Numerical	Price per unit.
Quantity	Numerical	Number of items purchased.
Date and Time	Temporal	Used to create month, weekday, weekend, and hour features.
Payment	Categorical	Cash, card, or e-wallet payment.
Rating	Numerical	Customer satisfaction score.
gross income	Numerical	Target variable predicted by the model.

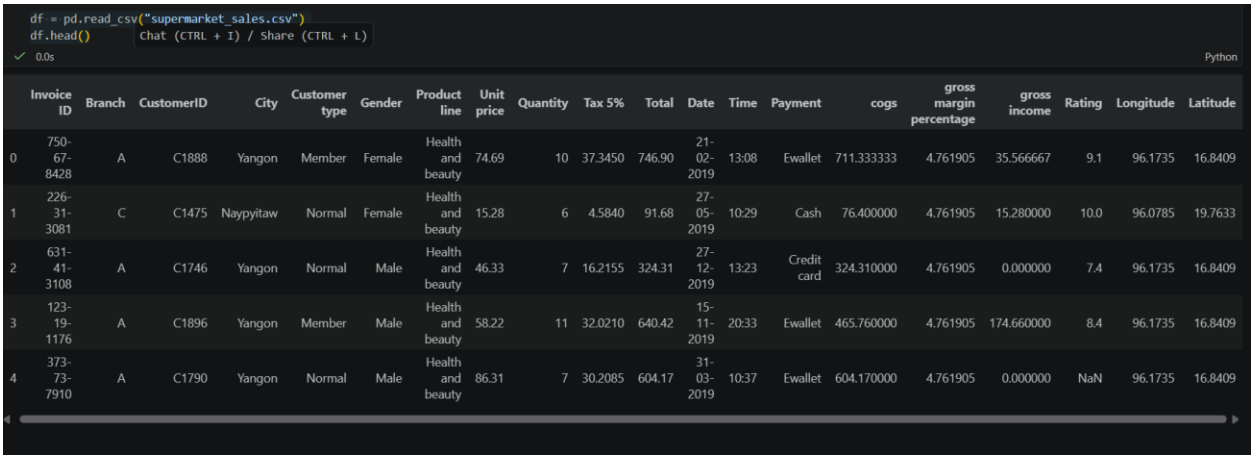
5. Exploratory Data Analysis

We conducted Exploratory Data Analysis in order to know about the structure of the dataset, analyze its completeness, distribution of variables, sales pattern over time, and correlation of variables to each other and to the target variable.

5.1 Dataset Summary

df = pd.read_csv("supermarket_sales.csv")

df.head()



```
df = pd.read_csv("supermarket_sales.csv")
df.head()        Chat (CTRL + I) / Share (CTRL + L)
```

0.0s Python

	Invoice ID	Branch	CustomerID	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating	Longitude	Latitude
0	750-67-8428	A	C1888	Yangon	Member	Female	Health and beauty	74.69	10	37.3450	746.90	21-02-2019	13:08	Ewallet	711.333333	4.761905	35.566667	9.1	96.1735	16.8409
1	226-31-3081	C	C1475	Naypyitaw	Normal	Female	Health and beauty	15.28	6	4.5840	91.68	27-05-2019	10:29	Cash	76.400000	4.761905	15.280000	10.0	96.0785	19.7633
2	631-41-3108	A	C1746	Yangon	Normal	Male	Health and beauty	46.33	7	16.2155	324.31	27-12-2019	13:23	Credit card	324.310000	4.761905	0.000000	7.4	96.1735	16.8409
3	123-19-1176	A	C1896	Yangon	Member	Male	Health and beauty	58.22	11	32.0210	640.42	15-11-2019	20:33	Ewallet	465.760000	4.761905	174.660000	8.4	96.1735	16.8409
4	373-73-7910	A	C1790	Yangon	Normal	Male	Health and beauty	86.31	7	30.2085	604.17	31-03-2019	10:37	Ewallet	604.170000	4.761905	0.000000	NaN	96.1735	16.8409

Figure 1: First Five Rows of the Supermarket Sales Dataset

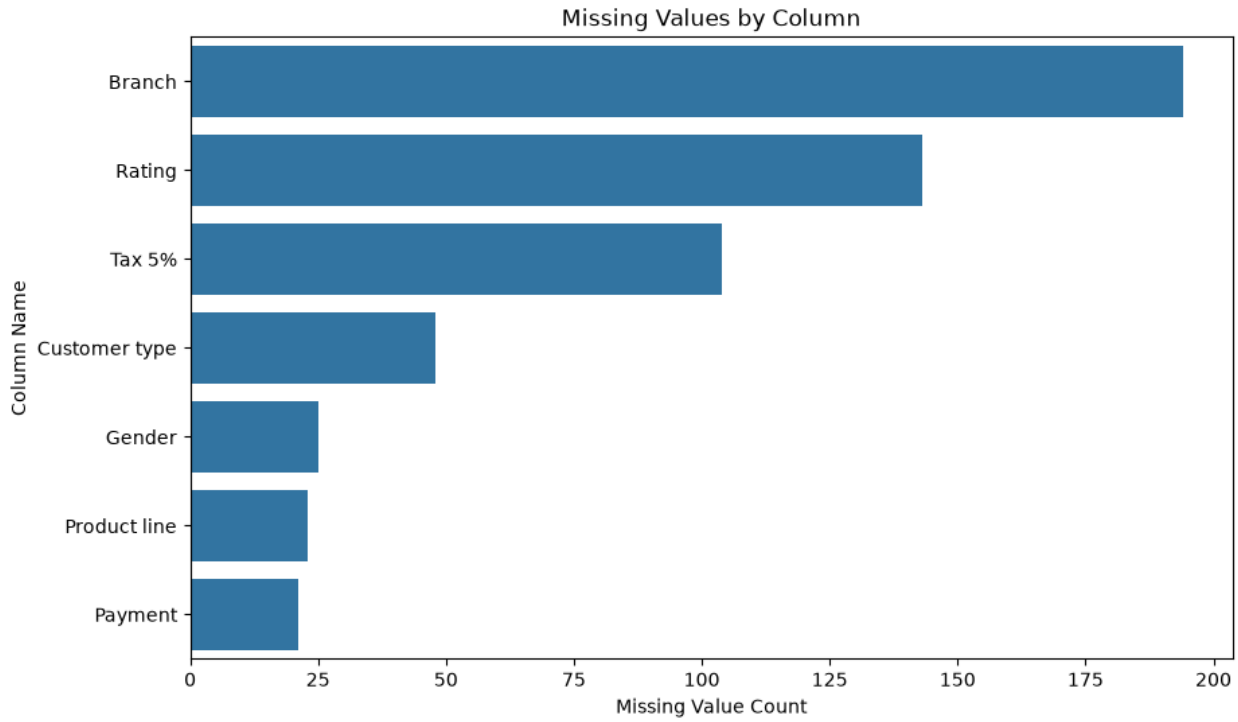
The dataset consists of both numeric and categorical variables, which implies that the preprocessing pipeline needs to deal with numeric imputation and scaling along with categorical

imputation and encodin

```
Dataset shape:
(1000, 20)

Column names:
Index(['Invoice ID', 'Branch', 'CustomerID', 'City', 'Customer type', 'Gender',
       'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Date',
       'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross income',
       'Rating', 'Longitude', 'Latitude'],
      dtype='str')

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 20 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Invoice ID              1000 non-null   str
 1   Branch                  806 non-null    str
 2   CustomerID              1000 non-null   str
 3   City                    1000 non-null   str
 4   Customer type           952 non-null    str
 5   Gender                  975 non-null    str
 6   Product line            977 non-null    str
 7   Unit price              1000 non-null   float64
 8   Quantity                1000 non-null   int64
...
 18  Longitude               1000 non-null   float64
 19  Latitude                1000 non-null   float64
dtypes: float64(9), int64(1), str(10)
memory usage: 156.4 KB
```

5.2 Dimensions and Missing Values

	Missing Count	Missing Percentage
Branch	194	19.4
Rating	143	14.3
Tax 5%	104	10.4
Customer type	48	4.8
Gender	25	2.5
Product line	23	2.3
Payment	21	2.1
Invoice ID	0	0.0
City	0	0.0
CustomerID	0	0.0
Unit price	0	0.0
Quantity	0	0.0
Date	0	0.0
Total	0	0.0
cogs	0	0.0
Time	0	0.0
gross margin percentage	0	0.0
gross income	0	0.0
Longitude	0	0.0
Latitude	0	0.0



Missing value analysis is done to find out whether Branch, Rating, Tax 5%, Customer type, Gender, Product line, and Payment have missing values. This was done during pre-processing where median imputation was done for numerical columns while most frequent value imputation was done for categorical columns.

```
missing_values = df.isnull().sum()
missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_table = pd.DataFrame({
    "Missing Count": missing_values,
    "Missing Percentage": missing_percentage
})

missing_table = missing_table.sort_values(by="Missing Count", ascending=False)

missing_table
```

5.2.1Missing values by column

```
import os
os.makedirs("eda_graphs", exist_ok=True)

missing_graph = missing_table[missing_table["Missing Count"] > 0]

plt.figure(figsize=(10, 6))

sns.barplot(
    x=missing_graph["Missing Count"],
    y=missing_graph.index
)

plt.title("Missing Values by Column")
plt.xlabel("Missing Value Count")
plt.ylabel("Column Name")

plt.savefig("eda_graphs/missing_values.png", dpi=300, bbox_inches="tight")
plt.show()
```

5.3 Statistical Summary

```
numerical_columns = df.select_dtypes(include=["int64", "float64"]).columns

df[numerical_columns].describe().T
```

	count	mean	std	min	25%	50%	75%	max
Unit price	1000.0	55.672130	26.494628	10.080000	32.875000	55.230000	77.935000	99.960000
Quantity	1000.0	7.485000	4.520643	1.000000	4.000000	7.000000	11.000000	20.000000
Tax 5%	896.0	20.966930	17.604704	0.508500	7.479500	15.468250	30.205875	87.498000
Total	1000.0	419.149340	347.824683	10.170000	152.745000	317.695000	605.222500	1749.960000
cogs	1000.0	307.775883	234.425682	10.170000	118.497500	241.760000	448.905000	993.000000
gross margin percentage	1000.0	4.761905	0.000000	4.761905	4.761905	4.761905	4.761905	4.761905
gross income	1000.0	111.373457	149.212835	0.000000	0.000000	63.650000	161.022500	874.980000
Rating	857.0	7.462625	1.776179	4.000000	5.900000	7.455000	9.100000	10.000000
Longitude	1000.0	96.114319	0.042715	96.078500	96.078500	96.089100	96.173500	96.173500
Latitude	1000.0	19.498590	2.106757	16.840900	16.840900	19.763300	21.958800	21.958800

Table 3: Statistical Summary of Important Numerical Columns

5.4 Univariate Analysis

Univariate analysis means analyzing one column at a time. In this section, the distribution of gross income and the count of important categorical columns are analyzed.

```
plt.figure(figsize=(10, 6))

sns.histplot(df["gross income"], kde=True)

plt.title("Distribution of Gross Income")
plt.xlabel("Gross Income")
plt.ylabel("Frequency")

plt.savefig("eda_graphs/gross_income_histogram.png", dpi=300,
bbox_inches="tight")
plt.show()
```

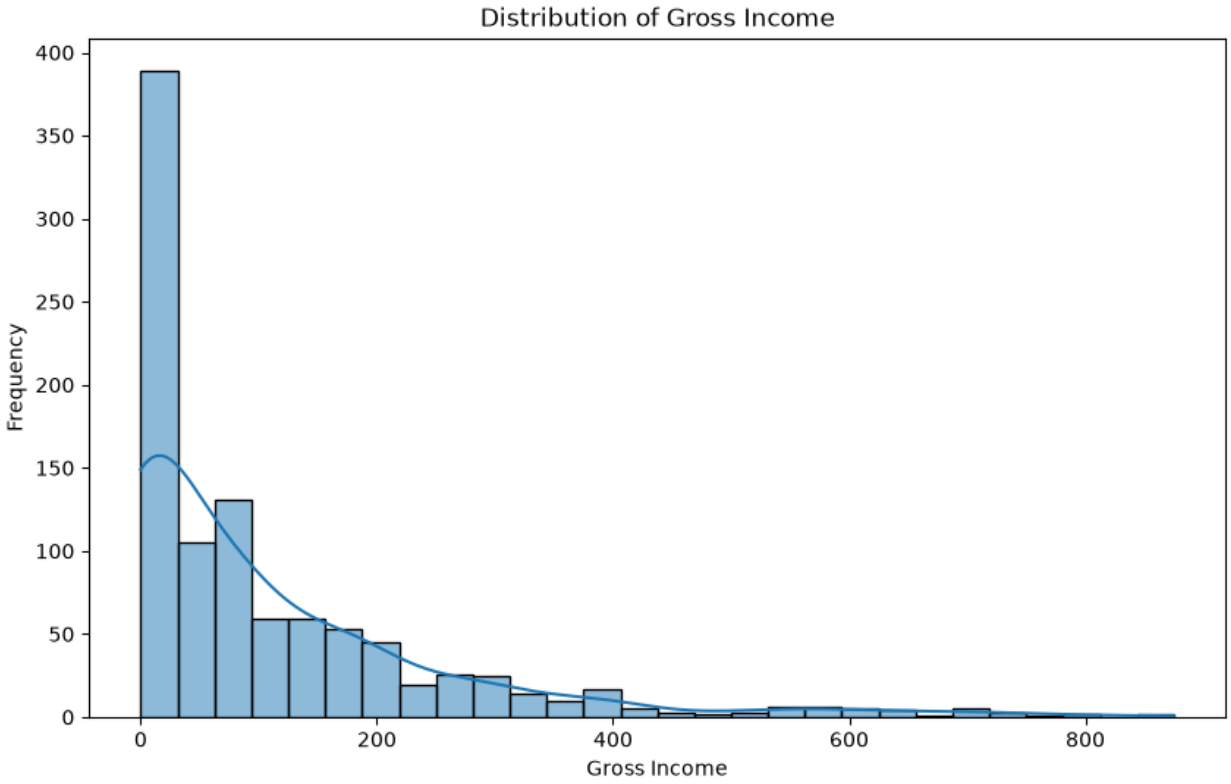


Figure 2: Distribution of Gross Income

```
plt.figure(figsize=(8, 5))

sns.boxplot(x=df["gross income"])

plt.title("Boxplot of Gross Income")
plt.xlabel("Gross Income")

plt.savefig("eda_graphs/gross_income_boxplot.png", dpi=300, bbox_inches="tight")
plt.show()
```

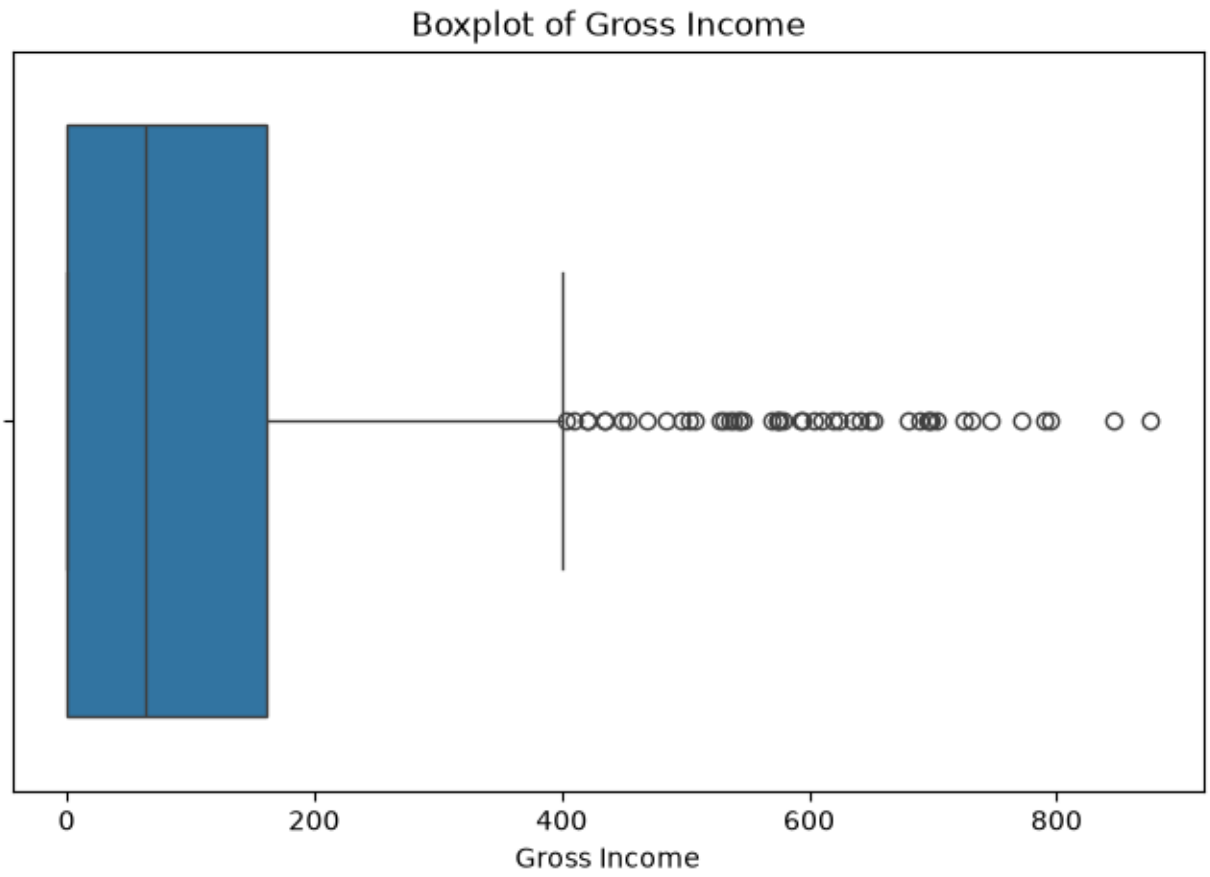


Figure 3: Boxplot of Gross Income

```
plt.figure(figsize=(12, 6))

sns.countplot(
    data=df,
    y="Product line",
    order=df["Product line"].value_counts().index
)

plt.title("Transaction Count by Product Line")
plt.xlabel("Number of Transactions")
plt.ylabel("Product Line")

plt.savefig("eda_graphs/product_line_count.png", dpi=300, bbox_inches="tight")
plt.show()
```

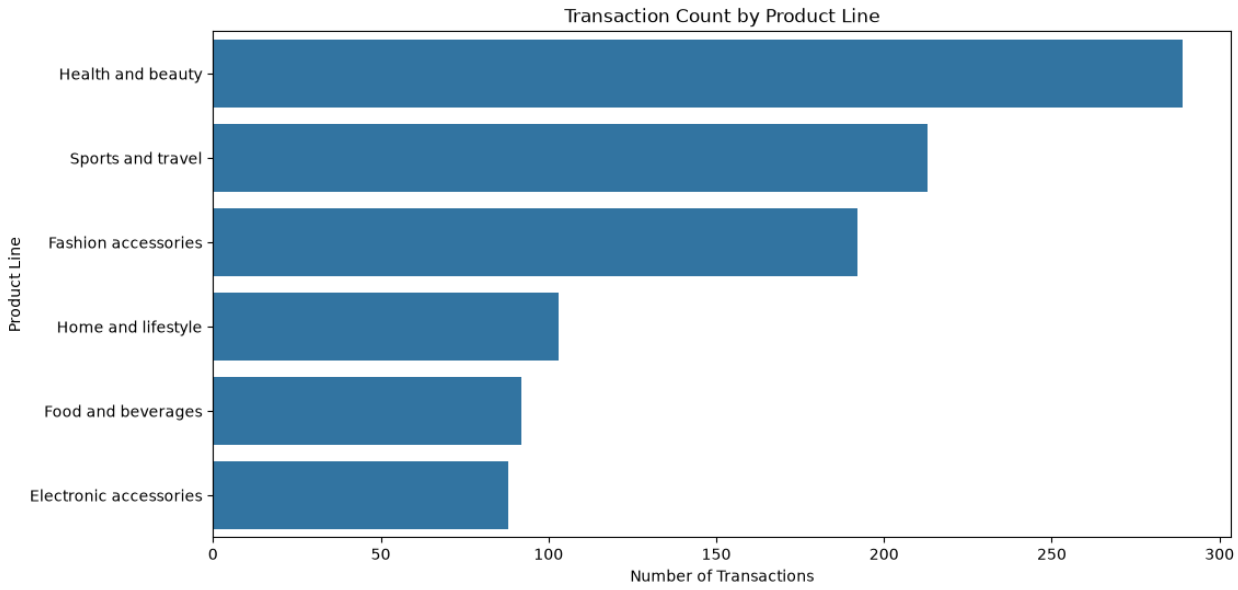


Figure 4: Transaction Count by Product Line

```
plt.figure(figsize=(8, 5))

sns.countplot(
    data=df,
    x="Payment",
    order=df["Payment"].value_counts().index
)

plt.title("Transaction Count by Payment Method")
plt.xlabel("Payment Method")
plt.ylabel("Number of Transactions")

plt.savefig("eda_graphs/payment_method_count.png", dpi=300, bbox_inches="tight")
plt.show()
```

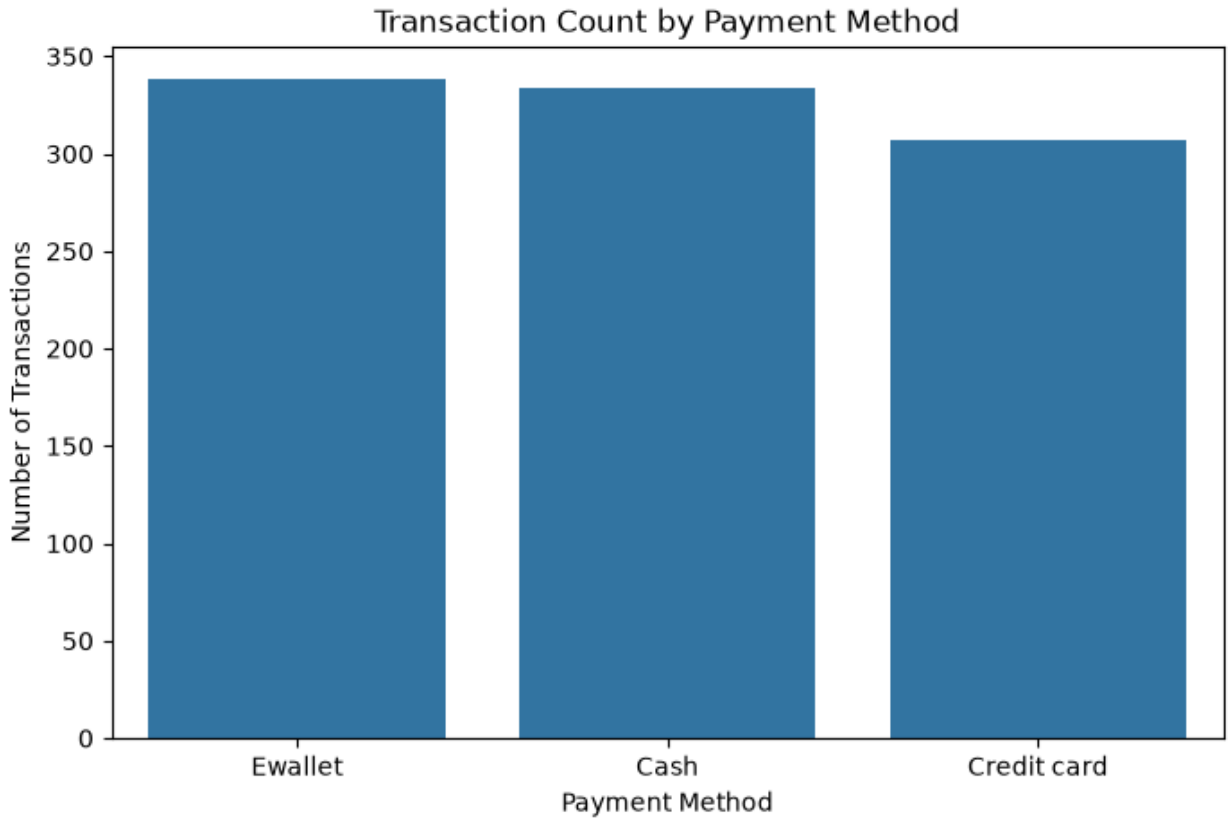


Figure 5: Transaction Count by Payment Method

5.4.1 Univariate Analysis Insight

The univariate analysis shows how individual columns are distributed. The gross income histogram shows the spread of income values. The boxplot helps to identify outliers. The product line and payment method charts show which categories appear more frequently in the dataset.

```
# Convert Date column to datetime format
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

# Create date-based features
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["DayOfWeek"] = df["Date"].dt.dayofweek
df["DayName"] = df["Date"].dt.day_name()

# Convert Time column and extract hour
```

```
df["Time"] = pd.to_datetime(df["Time"].astype(str), errors="coerce")
df["Hour"] = df["Time"].dt.hour

df[["Date", "Month", "DayName", "Hour"]].head()
```

	Date	Month	DayName	Hour
0	2019-02-21	2	Thursday	13
1	2019-05-27	5	Monday	10
2	2019-12-27	12	Friday	13
3	2019-11-15	11	Friday	20
4	2019-03-31	3	Sunday	10

```
daily_income = df.groupby("Date")["gross income"].sum().reset_index()

plt.figure(figsize=(12, 6))

sns.lineplot(data=daily_income, x="Date", y="gross income")

plt.title("Daily Gross Income Trend")
plt.xlabel("Date")
plt.ylabel("Total Gross Income")

plt.savefig("eda_graphs/daily_gross_income_trend.png", dpi=300,
bbox_inches="tight")
plt.show()
```

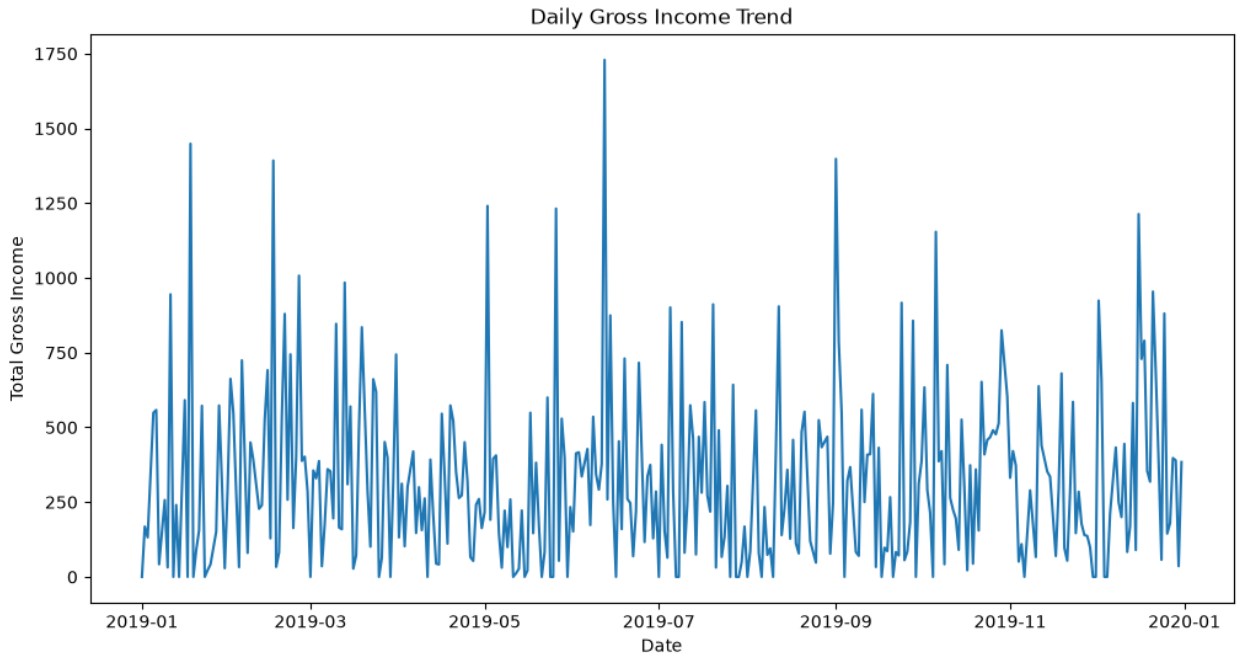


Figure 6: Daily Gross Income Trend

```
monthly_income = df.groupby("Month")["gross income"].sum().reset_index()

plt.figure(figsize=(8, 5))

sns.lineplot(data=monthly_income, x="Month", y="gross income", marker="o")

plt.title("Monthly Gross Income Trend")
plt.xlabel("Month")
plt.ylabel("Total Gross Income")

plt.savefig("eda_graphs/monthly_gross_income_trend.png", dpi=300,
bbox_inches="tight")
plt.show()
```

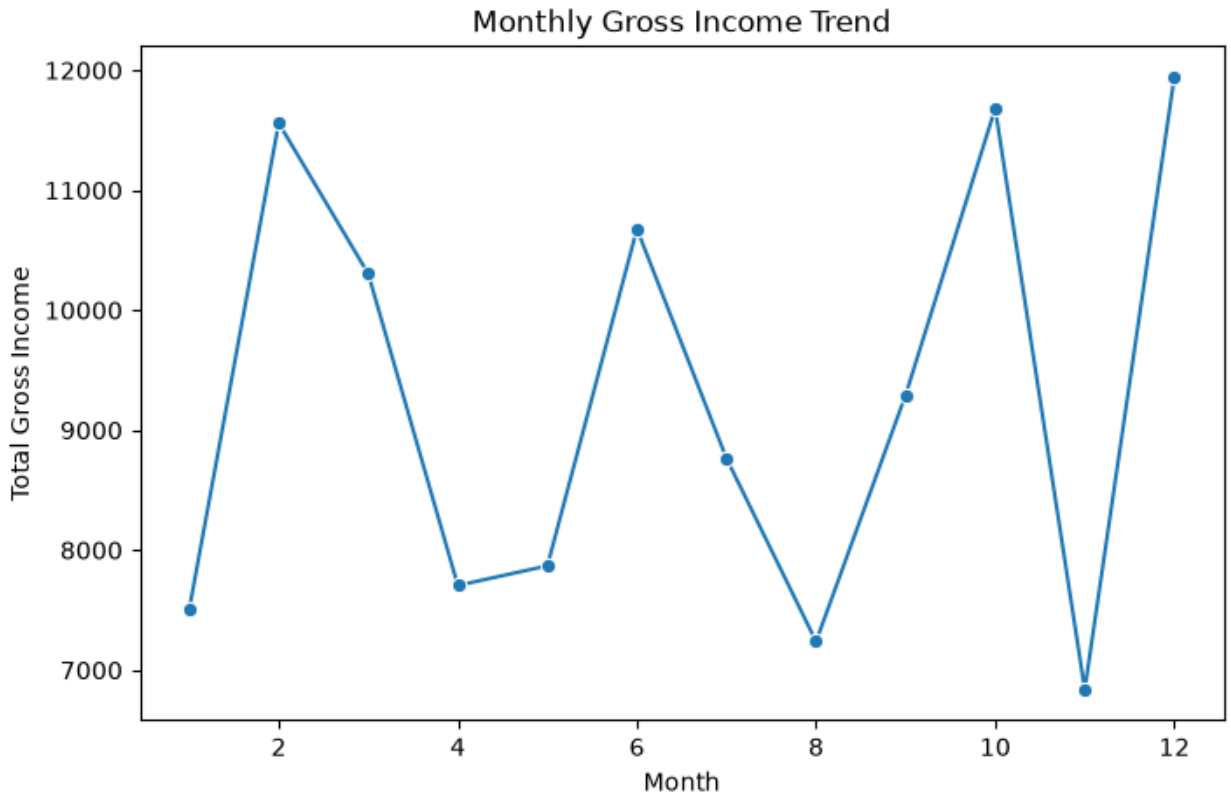


Figure 7: Monthly Gross Income Trend

```
day_order = [
    "Monday", "Tuesday", "Wednesday", "Thursday",
    "Friday", "Saturday", "Sunday"
]

day_income = df.groupby("DayName")["gross
income"].mean().reindex(day_order).reset_index()

plt.figure(figsize=(10, 5))

sns.barplot(data=day_income, x="DayName", y="gross income")

plt.title("Average Gross Income by Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Average Gross Income")
plt.xticks(rotation=45)

plt.savefig("eda_graphs/average_gross_income_by_day.png", dpi=300,
bbox_inches="tight")
plt.show()
```

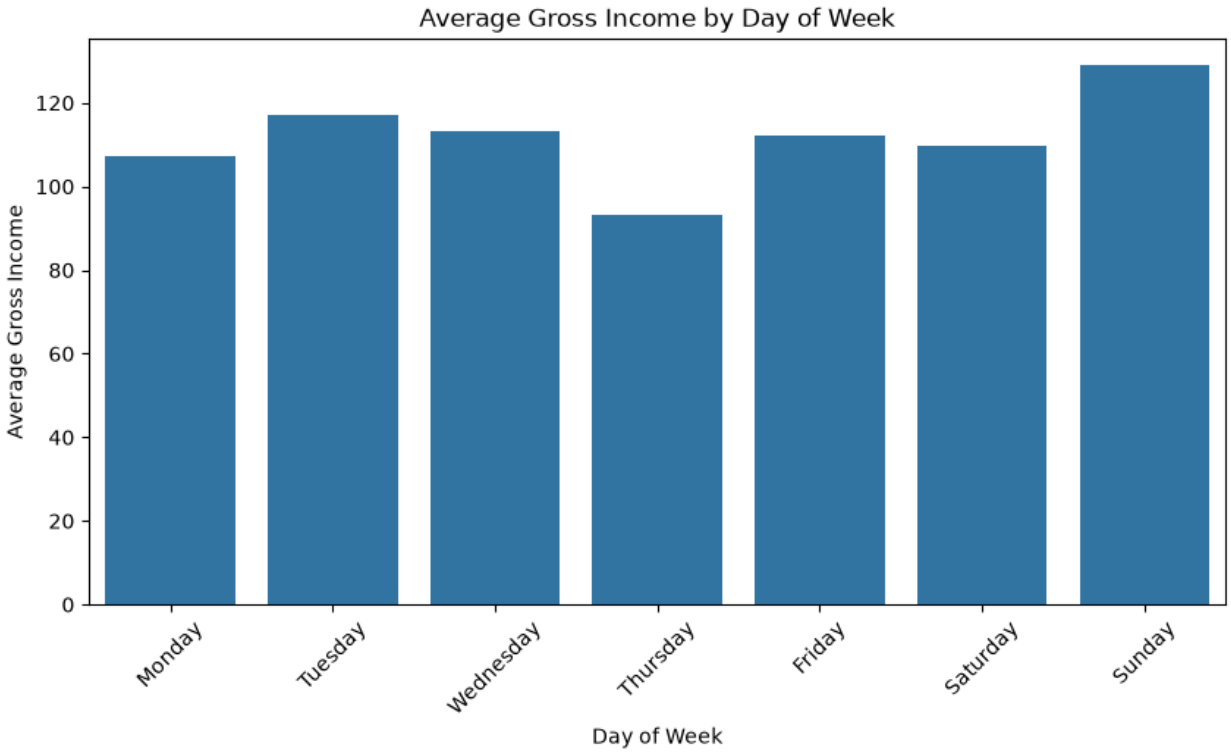


Figure 8: Average Gross Income by Day of Week

```
hour_income = df.groupby("Hour")["gross income"].mean().reset_index()

plt.figure(figsize=(10, 5))

sns.lineplot(data=hour_income, x="Hour", y="gross income", marker="o")

plt.title("Average Gross Income by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Average Gross Income")

plt.savefig("eda_graphs/average_gross_income_by_hour.png", dpi=300,
bbox_inches="tight")
plt.show()
```

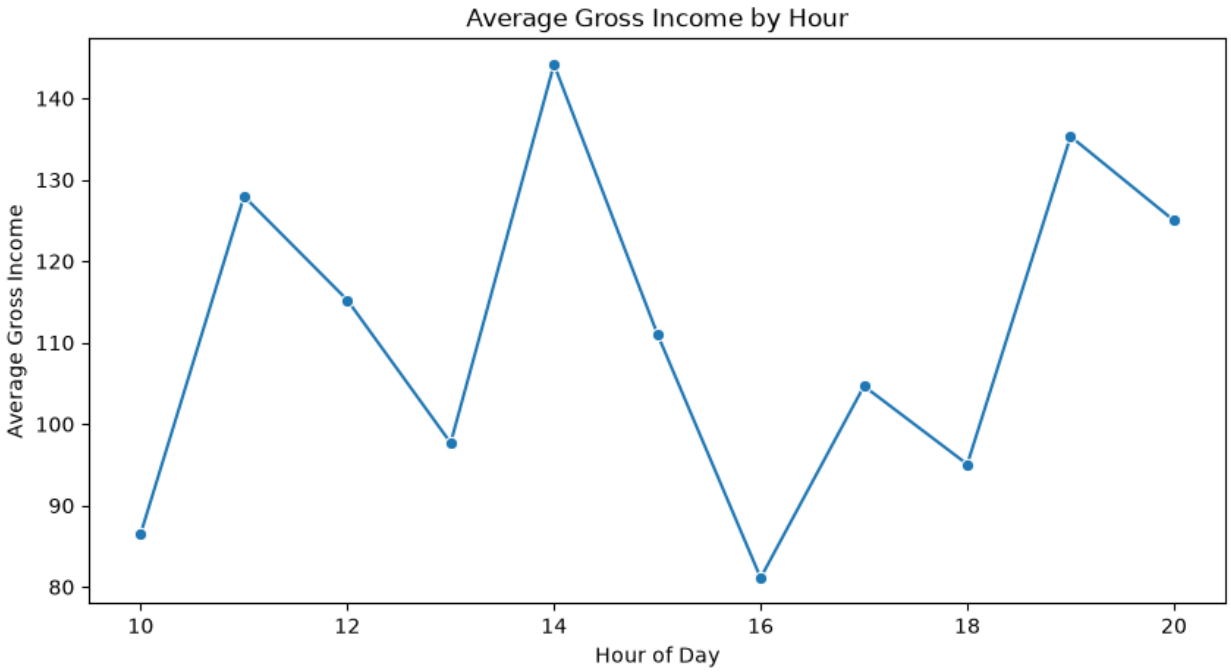


Figure 9: Average Gross Income by Hour

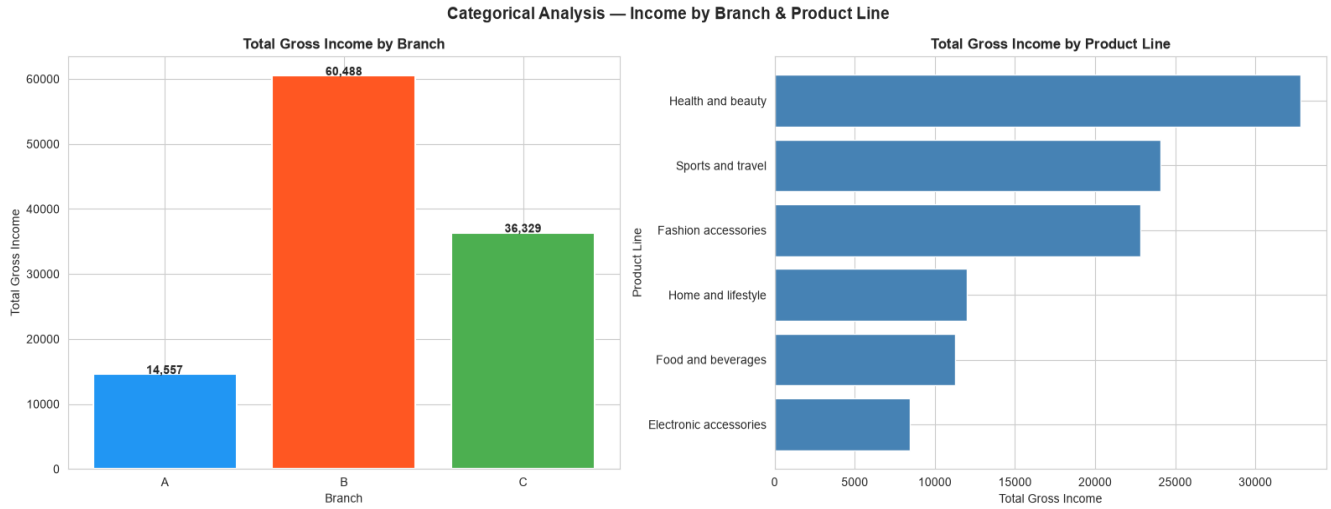
From the time series graphs, there is variability in the sale patterns depending on the month, day, and hour. Consequently, the following features were engineered: Month, DayOfWeek, DayName, IsWeekend, and Hour.

Categorical Analysis

Sales performance across branches and product lines

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Chart 1 — Gross Income by Branch
branch_income = df.groupby('Branch')['gross income'].sum().reset_index()
colors_branch = ['#2196F3', '#FF5722', '#4CAF50']
axes[0].bar(branch_income['Branch'], branch_income['gross income'],
            color=colors_branch, edgecolor='white', linewidth=1.5)
axes[0].set_title('Total Gross Income by Branch', fontweight='bold')
axes[0].set_xlabel('Branch')
axes[0].set_ylabel('Total Gross Income')
for i, v in enumerate(branch_income['gross income']):
    axes[0].text(i, v + 50, f'{v:,.0f}', ha='center', fontweight='bold')

# Chart 2 — Gross Income by Product Line
```

```
product_income = df.groupby('Product line')['gross income'].sum().sort_values()
axes[1].barh(product_income.index, product_income.values,
             color='steelblue', edgecolor='white')
axes[1].set_title('Total Gross Income by Product Line', fontweight='bold')
axes[1].set_xlabel('Total Gross Income')
axes[1].set_ylabel('Product Line')

plt.suptitle('Categorical Analysis — Income by Branch & Product Line',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Key EDA Finding — Branch Income Gap Explained

Branch B generates the highest total gross income (60,488) compared to Branch C (36,329) and Branch A (14,557). Investigation revealed this is NOT due to pricing differences (unit prices are nearly identical: A=54.78, B=55.66, C=56.61) but due to **quantity per transaction**:

- Branch A avg quantity: 6.26 items/transaction

- Branch B avg quantity: 8.75 items/transaction

- Branch C avg quantity: 7.47 items/transaction

Branch B customers consistently purchase more items per visit, driving higher gross income per transaction.

Business Recommendation: Branch A should investigate upselling strategies to increase items per transaction.

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Chart 1 — Corrected Gross Income by Branch
branch_income = df.groupby('Branch')['gross income'].sum().reset_index()
colors_branch = ['#2196F3', '#FF5722', '#4CAF50']
bars = axes[0].bar(branch_income['Branch'], branch_income['gross income'],
                   color=colors_branch, edgecolor='white', linewidth=1.5)
axes[0].set_title('Total Gross Income by Branch\n(After Data Correction)',
                  fontweight='bold')
axes[0].set_xlabel('Branch')
axes[0].set_ylabel('Total Gross Income')
for i, v in enumerate(branch_income['gross income']):
    axes[0].text(i, v + 100, f'{v:,.0f}', ha='center', fontweight='bold')

# Chart 2 — Average Quantity by Branch (explains the gap)
branch_qty = df.groupby('Branch')['Quantity'].mean().reset_index()
axes[1].bar(branch_qty['Branch'], branch_qty['Quantity'],
            color=colors_branch, edgecolor='white', linewidth=1.5)
axes[1].set_title('Average Quantity per Transaction by Branch\n(Root Cause of
Income Gap)',
                  fontweight='bold')
axes[1].set_xlabel('Branch')
axes[1].set_ylabel('Average Quantity')
for i, v in enumerate(branch_qty['Quantity']):
    axes[1].text(i, v + 0.05, f'{v:.2f}', ha='center', fontweight='bold')

plt.suptitle('Branch Analysis — Income Gap Root Cause Investigation',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

5.5 Temporal Analysis

Sales trends over time — identifying patterns by date, day of week, and hour

5.6 Correlation and Relationship Analysis

Correlation and relationship analysis is used to understand how numerical features are connected with the target variable. This helps to identify which features may affect gross income.

```
numerical_columns = df.select_dtypes(include=["int64", "float64"]).columns

correlation_matrix = df[numerical_columns].corr()

plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Numerical Features")

plt.savefig("eda_graphs/correlation_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()
```

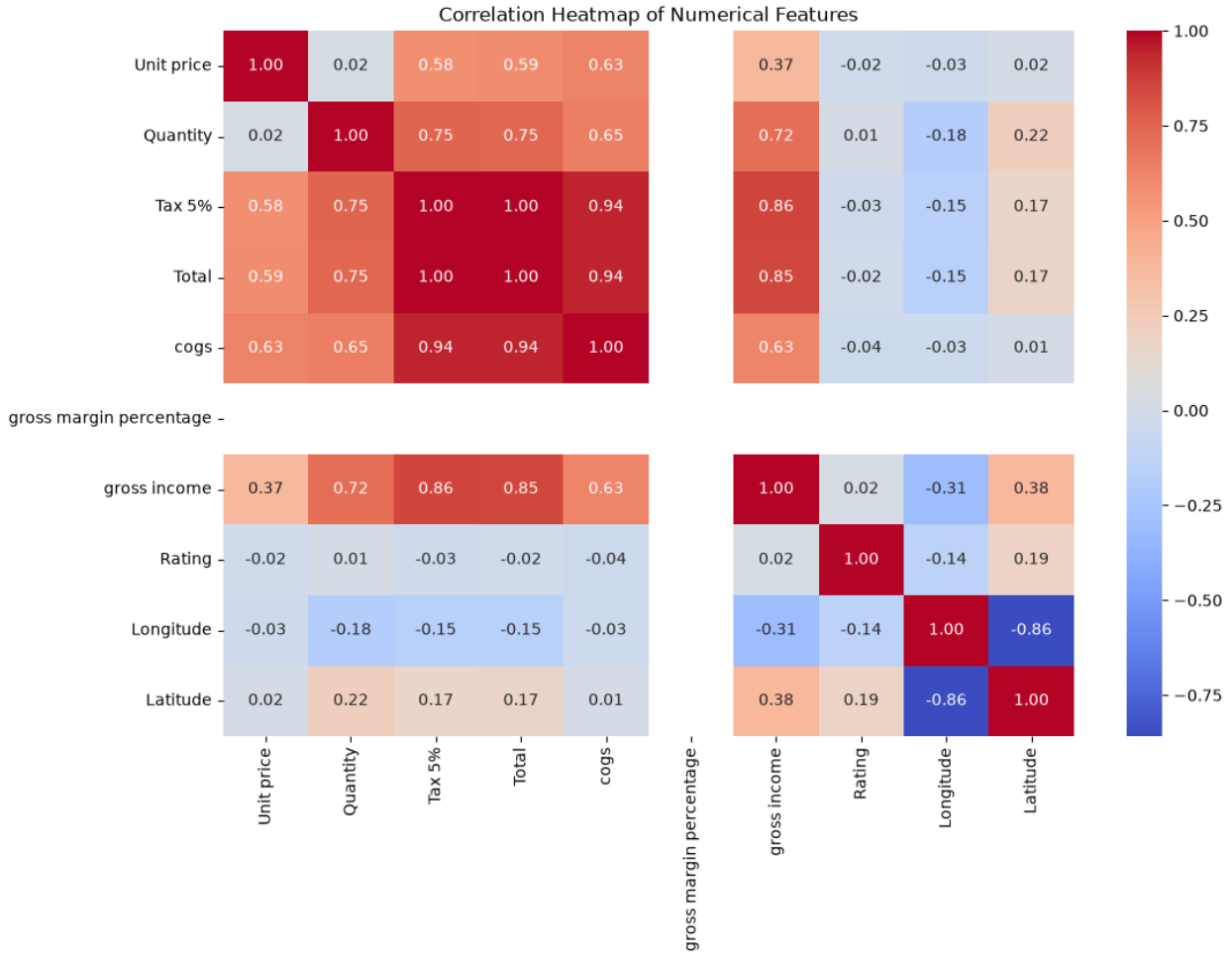


Figure 10: Correlation Heatmap of Numerical Features

```
plt.figure(figsize=(8, 5))

sns.scatterplot(data=df, x="Total", y="gross income")

plt.title("Relationship Between Total and Gross Income")
plt.xlabel("Total")
plt.ylabel("Gross Income")

plt.savefig("eda_graphs/total_vs_gross_income.png", dpi=300, bbox_inches="tight")
plt.show()
```

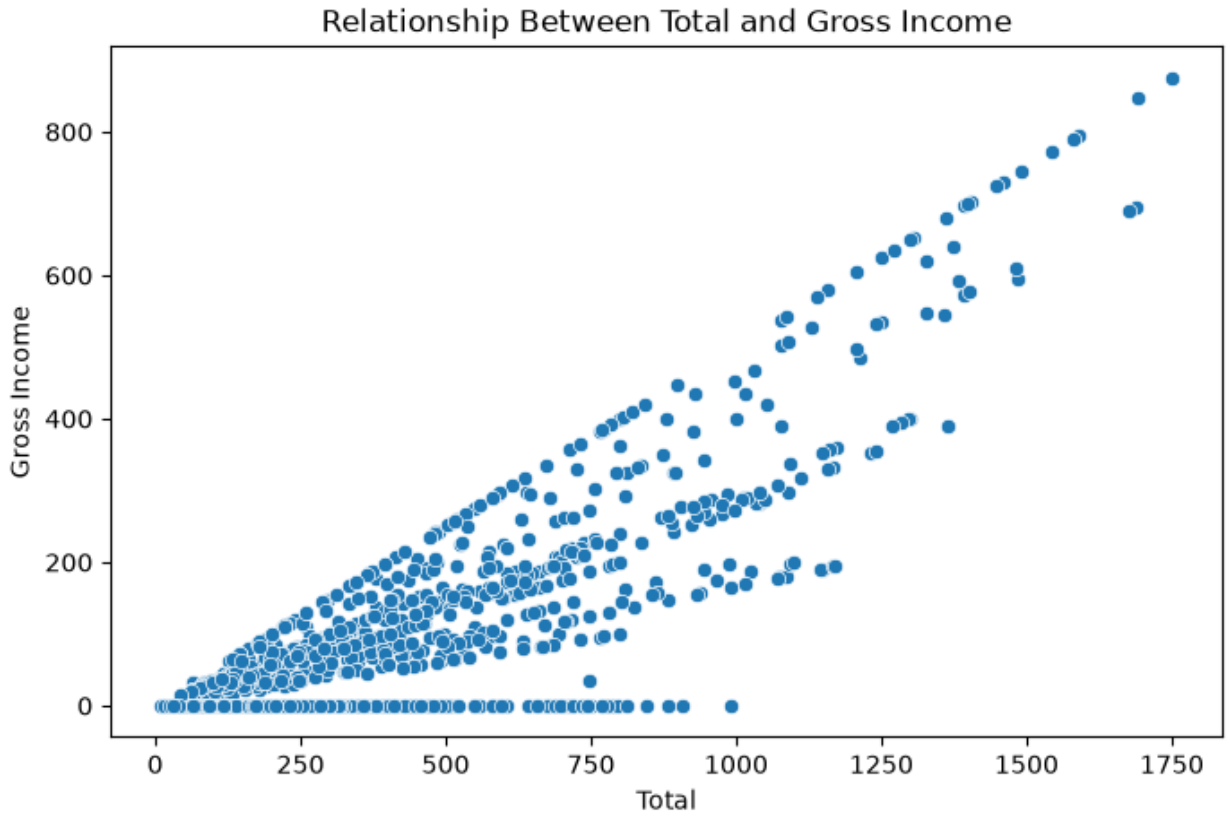


Figure 11: Total vs Gross Income

```
plt.figure(figsize=(8, 5))

sns.scatterplot(data=df, x="Quantity", y="gross income")

plt.title("Relationship Between Quantity and Gross Income")
plt.xlabel("Quantity")
```

```
plt.ylabel("Gross Income")

plt.savefig("eda_graphs/quantity_vs_gross_income.png", dpi=300,
bbox_inches="tight")
plt.show()
```

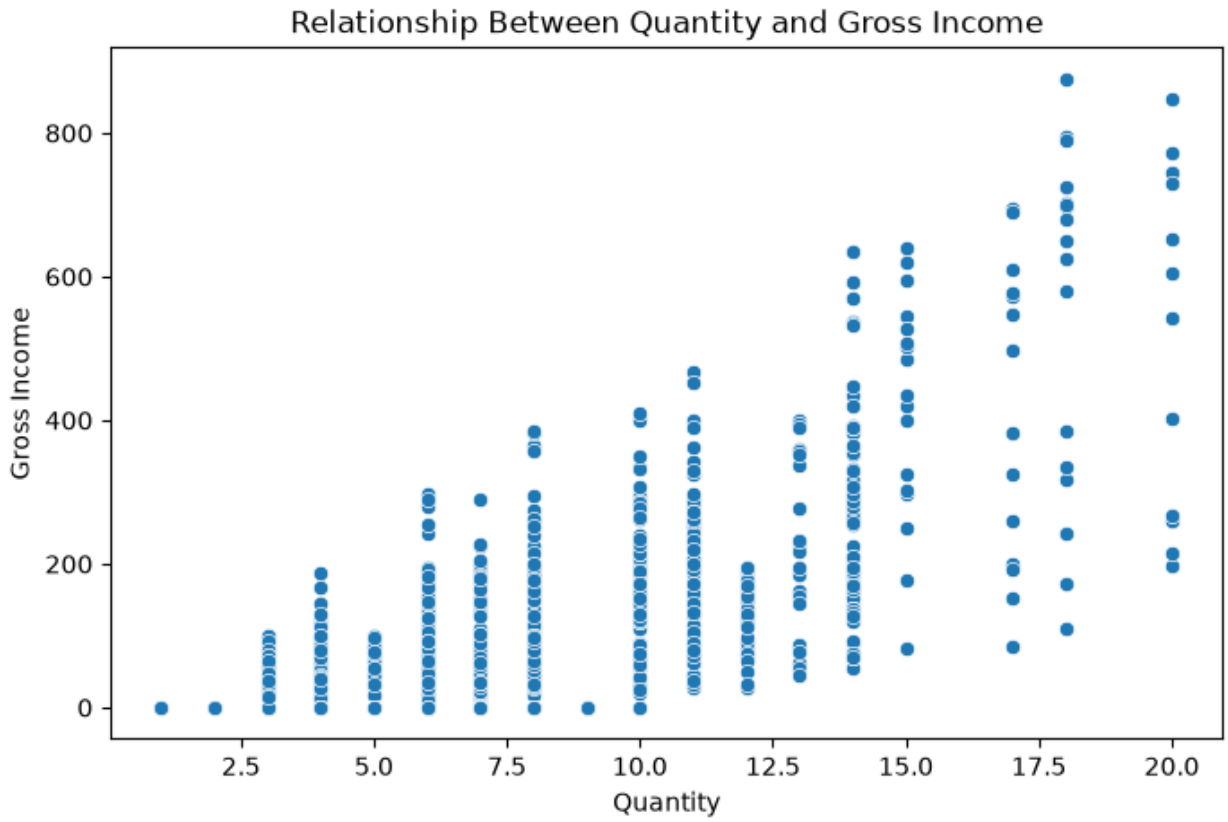


Figure 12: Quantity vs Gross Income

```
plt.figure(figsize=(8, 5))

sns.scatterplot(data=df, x="Unit price", y="gross income")

plt.title("Relationship Between Unit Price and Gross Income")
plt.xlabel("Unit Price")
plt.ylabel("Gross Income")

plt.savefig("eda_graphs/unit_price_vs_gross_income.png", dpi=300,
bbox_inches="tight")
plt.show()
```

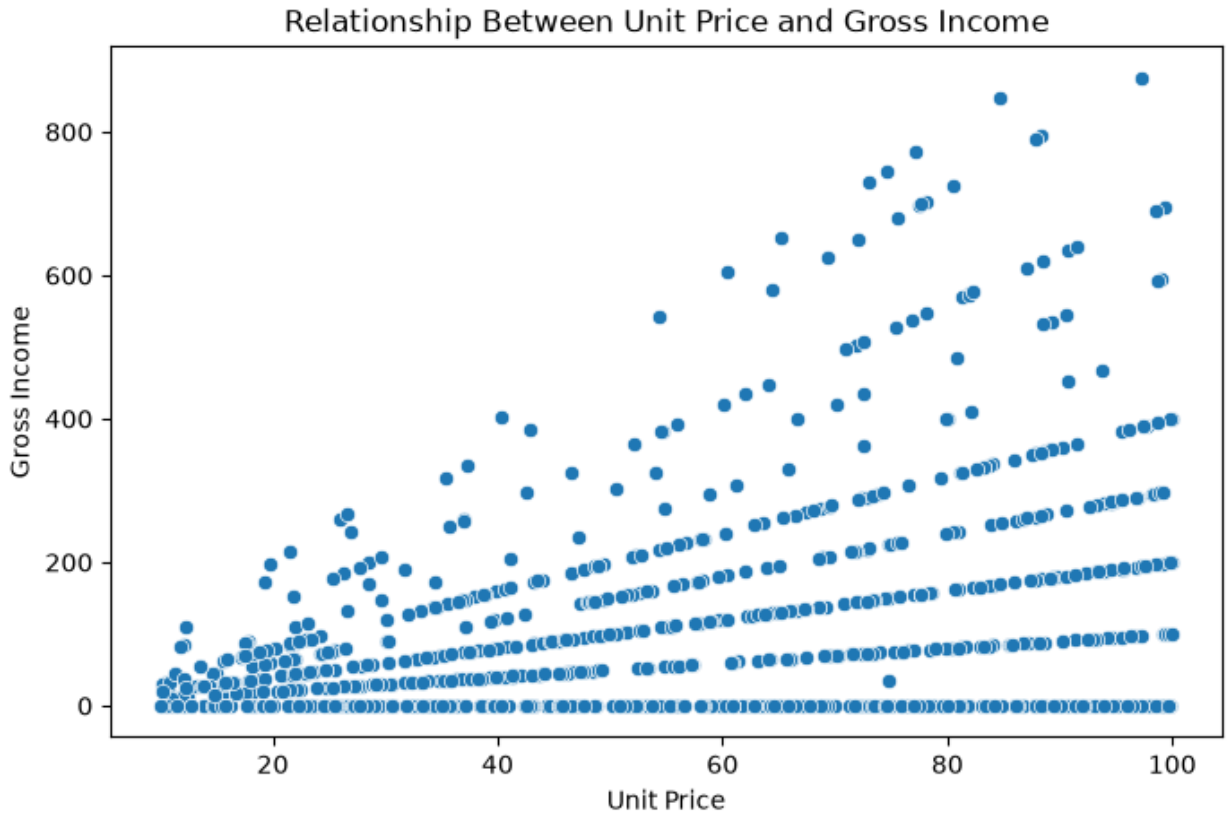


Figure 13: Unit Price vs Gross Income

```
plt.figure(figsize=(12, 6))

sns.boxplot(data=df, x="gross income", y="Product line")

plt.title("Gross Income by Product Line")
plt.xlabel("Gross Income")
plt.ylabel("Product Line")

plt.savefig("eda_graphs/gross_income_by_product_line.png", dpi=300,
bbox_inches="tight")
plt.show()
```

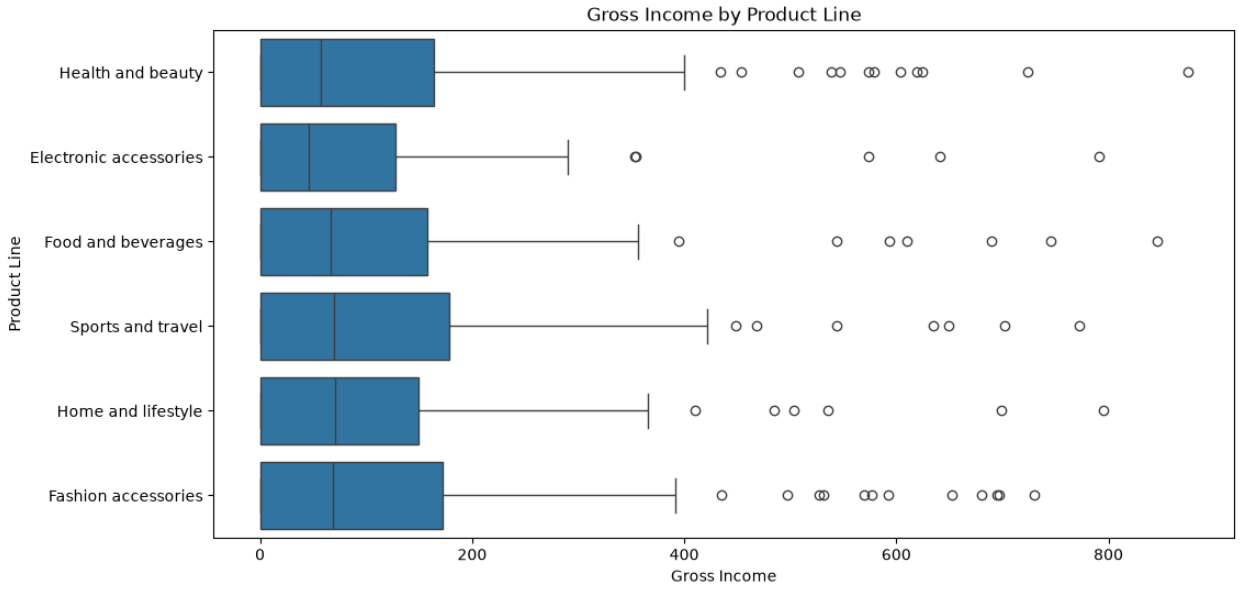


Figure 14: Gross Income by Product Line

```
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Branch", y="gross income")

plt.title("Gross Income by Branch")
plt.xlabel("Branch")
plt.ylabel("Gross Income")

plt.savefig("eda_graphs/gross_income_by_branch.png", dpi=300,
bbox_inches="tight")
plt.show()
```

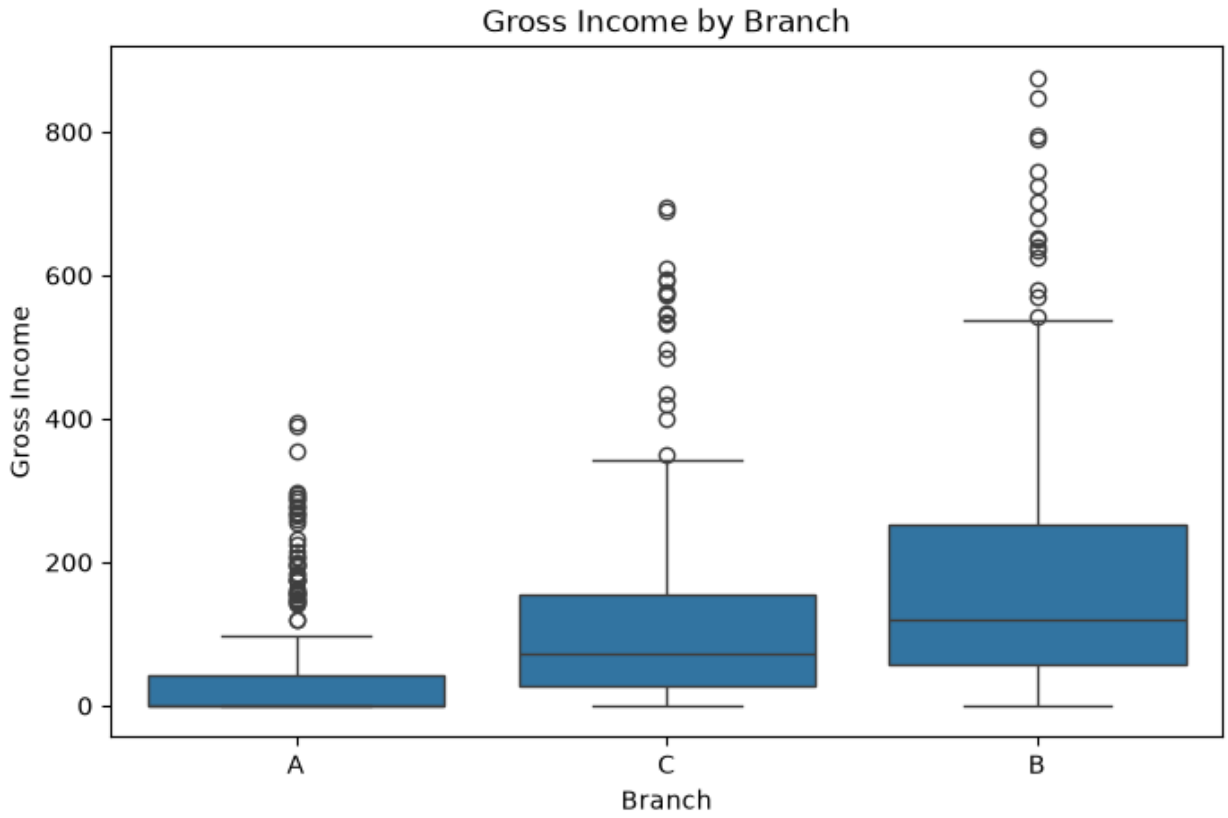


Figure 15: Gross Income by Branch

From the heat map, Total, Tax 5%, Quantity, and Unit price have high correlations with gross income. Nonetheless, Total, Tax 5%, cogs, and gross margin percentage were not included in the input variables for the model due to their high correlation with the target variable.

Correlation and Relationship Analysis Insight

The correlation heatmap shows the relationships between numerical columns. Features such as Total, Tax 5%, Quantity, and Unit price are important because they can affect gross income. The scatter plots show how gross income changes when Total, Quantity, and Unit price change. The boxplots show that gross income can also differ based on product line and branch.

```
import os

print("Saved EDA graph files:")

for file in os.listdir("eda_graphs"):
    print(file)
```

6. Data Preprocessing Stage

During the preprocessing stage, the raw dataset is transformed into a clean dataset that can be used for modeling purposes. This step matters because machine learning methods cannot learn from missing values directly, textual features, or non-processed date/time fields.

```
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

os.makedirs("model_graphs", exist_ok=True)

print("ML libraries imported successfully")
```

6.1 Missing Value Handling

Missing values were handled separately for numerical and categorical columns. Numerical columns were filled using the median value. Categorical columns were filled using the most frequent value.

```
# Make a copy of the dataset for preprocessing
df_model = df.copy()

# Check missing values before preprocessing
df_model.isnull().sum()
```

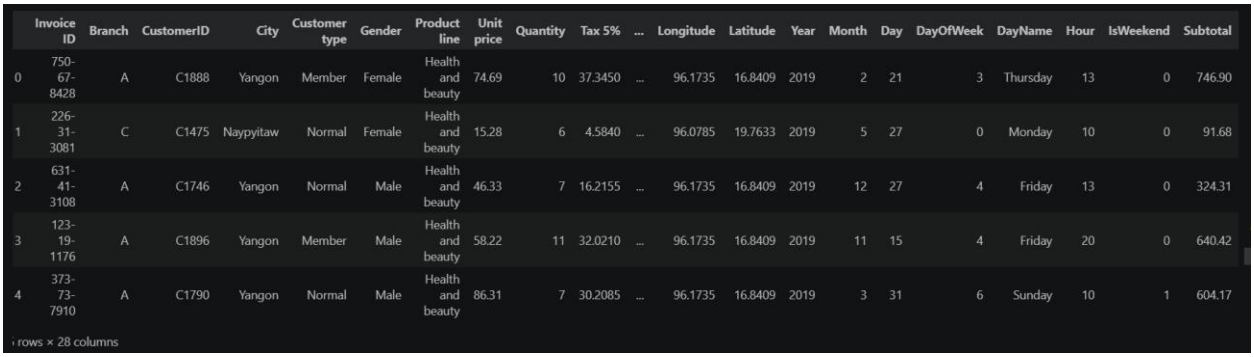
```
Invoice ID                  0
Branch                    194
CustomerID                  0
City                        0
Customer type              48
Gender                     25
Product line               23
Unit price                  0
Quantity                    0
Tax 5%                    104
Total                       0
Date                        0
Time                        0
Payment                    21
cogs                        0
gross margin percentage     0
gross income                0
Rating                    143
Longitude                   0
Latitude                    0
Year                        0
Month                       0
Day                         0
DayOfWeek                   0
DayName                     0
Hour                        0
dtype: int64
```

6.2Feature Engineering

New time-based and sales-related features were created to help the model learn better patterns from the dataset.

```
# Convert Date column to datetime
df_model["Date"] = pd.to_datetime(df_model["Date"], errors="coerce")

# Convert Time column
df_model["Time"] = pd.to_datetime(df_model["Time"].astype(str), errors="coerce")

# Create date and time features
df_model["Year"] = df_model["Date"].dt.year
```

```
df_model["Month"] = df_model["Date"].dt.month
df_model["Day"] = df_model["Date"].dt.day
df_model["DayOfWeek"] = df_model["Date"].dt.dayofweek
df_model["IsWeekend"] = df_model["DayOfWeek"].apply(lambda x: 1 if x >= 5 else 0)
df_model["Hour"] = df_model["Time"].dt.hour

# Create simple sales feature
if "Unit price" in df_model.columns and "Quantity" in df_model.columns:
    df_model["Subtotal"] = df_model["Unit price"] * df_model["Quantity"]

df_model.head()
```



	Invoice ID	Branch	CustomerID	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	...	Longitude	Latitude	Year	Month	Day	DayOfWeek	DayName	Hour	IsWeekend	Subtotal
0	750-67-8428	A	C1888	Yangon	Member	Female	Health and beauty	74.69	10	37.3450	...	96.1735	16.8409	2019	2	21	3	Thursday	13	0	746.90
1	226-31-3081	C	C1475	Naypyitaw	Normal	Female	Health and beauty	15.28	6	4.5840	...	96.0785	19.7633	2019	5	27	0	Monday	10	0	91.68
2	631-41-3108	A	C1746	Yangon	Normal	Male	Health and beauty	46.33	7	16.2155	...	96.1735	16.8409	2019	12	27	4	Friday	13	0	324.31
3	123-19-1176	A	C1896	Yangon	Member	Male	Health and beauty	58.22	11	32.0210	...	96.1735	16.8409	2019	11	15	4	Friday	20	0	640.42
4	373-73-7910	A	C1790	Yangon	Normal	Male	Health and beauty	86.31	7	30.2085	...	96.1735	16.8409	2019	3	31	6	Sunday	10	1	604.17

rows × 28 columns

6.3 Encoding and Scaling

The categorical variables such as the branch, city, customer type, gender, product line, and payment were encoded using the One-Hat Encoding method. The numerical variables were imputed using their median values. Scaling was utilized in the Linear Regression model but this was not the case in the tree-based models especially because the Random Forest and XGBoost models do not require any scaling.

6.4Train, Validation, and Test Split

A temporal split was used instead of a random split. The dataset was sorted by date first. The first 70% of records were used for training, the next 15% for validation, and the final 15% for testing. This helps to reduce data leakage.

```
# Sort dataset by Date for temporal splitting
df_model = df_model.sort_values("Date").reset_index(drop=True)
```

```
# Target variable
target = "gross income"

# Remove columns that should not be used as input features
drop_columns = [
    target,
    "Invoice ID",
    "Date",
    "Time",
    "Tax 5%",
    "Total",
    "cogs",
    "gross margin percentage"
]

# Keep only columns that exist
drop_columns = [col for col in drop_columns if col in df_model.columns]

X = df_model.drop(columns=drop_columns)
y = df_model[target]

print("Input features:")
print(X.columns)

print("\nTarget variable:")
print(target)
```

Input features:
Index(['Branch', 'CustomerID', 'City', 'Customer type', 'Gender',
'Product line', 'Unit price', 'Quantity', 'Payment', 'Rating',
'Longitude', 'Latitude', 'Year', 'Month', 'Day', 'DayOfWeek', 'DayName',
'Hour', 'IsWeekend', 'Subtotal'],
dtype='str')

Target variable:
gross income

```
n = len(df_model)

train_end = int(n * 0.70)
val_end = int(n * 0.85)

X_train = X.iloc[:train_end]
y_train = y.iloc[:train_end]
```

```
X_val = X.iloc[train_end:val_end]
y_val = y.iloc[train_end:val_end]

X_test = X.iloc[val_end:]
y_test = y.iloc[val_end:]

split_table = pd.DataFrame({
    "Dataset": ["Training Set", "Validation Set", "Test Set"],
    "Percentage": ["70%", "15%", "15%"],
    "Number of Records": [len(X_train), len(X_val), len(X_test)],
    "Purpose": [
        "Used to train the models",
        "Used to compare baseline models",
        "Used for final tuned model evaluation"
    ]
})

split_table
```

	Dataset	Percentage	Number of Records	Purpose
0	Training Set	70%	700	Used to train the models
1	Validation Set	15%	150	Used to compare baseline models
2	Test Set	15%	150	Used for final tuned model evaluation

6.5Preprocessing Pipeline Code Summary

A reusable preprocessing pipeline was created using scikit-learn. Numerical columns were imputed and scaled. Categorical columns were imputed and encoded using One-Hot Encoding.

```
# Identify numerical and categorical columns
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["object"]).columns

print("Numerical features:")
print(numeric_features)

print("\nCategorical features:")
```

```
print(categorical_features)
```

```
# Numerical preprocessing
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical preprocessing
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Full preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

print("Preprocessing pipeline created successfully")
```

7.Model Selection and Baseline Implementation

Three regression algorithms were selected for baseline comparison: Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor.

7.1 Algorithms Used

1. Linear Regression was used as a simple baseline model.

2. Random Forest Regressor was used because it can handle non-linear relationships and feature interactions.

3. Gradient Boosting Regressor was used because it improves prediction by learning from previous errors.

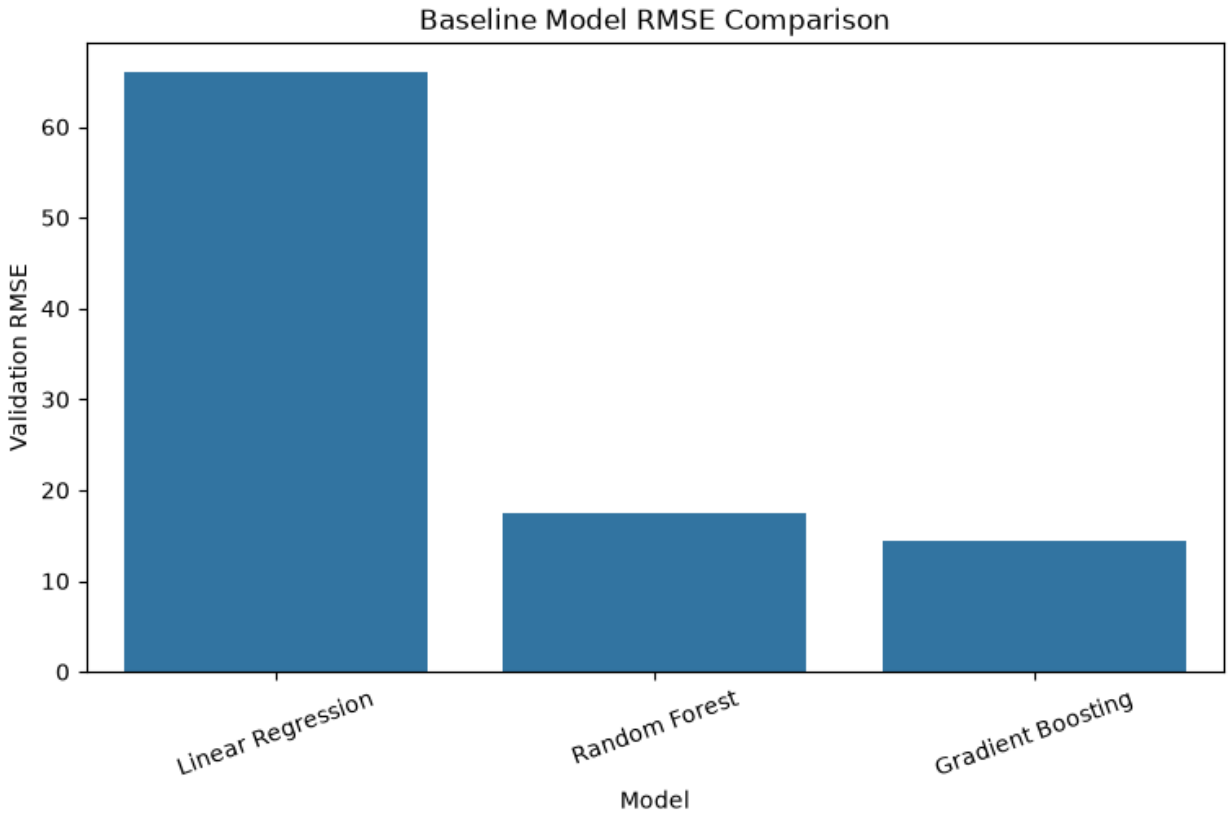
```
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(random_state=42, n_estimators=100),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42)
}

baseline_results = []

trained_models = {}

for model_name, model in models.items():
    pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", model)
    ])

    pipeline.fit(X_train, y_train)
    y_val_pred = pipeline.predict(X_val)

    rmse = np.sqrt(mean_squared_error(y_val, y_val_pred))
    mae = mean_absolute_error(y_val, y_val_pred)
    r2 = r2_score(y_val, y_val_pred)

    baseline_results.append({
        "Model": model_name,
        "Validation RMSE": rmse,
        "Validation MAE": mae,
        "Validation R2": r2
    })

    trained_models[model_name] = pipeline

baseline_results_df = pd.DataFrame(baseline_results)
baseline_results_df
```

	Model	Validation RMSE	Validation MAE	Validation R2
0	Linear Regression	65.942537	49.754019	0.827346
1	Random Forest	17.437204	9.506693	0.987927
2	Gradient Boosting	14.461794	10.398318	0.991696

7.2 Baseline Validation Results

The baseline models were trained using default hyperparameters. The validation set was used to compare model performance using RMSE, MAE, and R^2 score.

```
plt.figure(figsize=(9, 5))

sns.barplot(
    data=baseline_results_df,
    x="Model",
    y="Validation RMSE"
)

plt.title("Baseline Model RMSE Comparison")
plt.xlabel("Model")
plt.ylabel("Validation RMSE")
plt.xticks(rotation=20)

plt.savefig("model_graphs/baseline_rmse_comparison.png", dpi=300,
bbox_inches="tight")
plt.show()
```



7.3 Model Training Code Summary

Each model was trained using the same preprocessing pipeline. This makes the model training process consistent and reduces manual preprocessing errors. The best model was selected based on the lowest validation RMSE and highest validation R^2 score.

```
best_model_name = baseline_results_df.sort_values("Validation
RMSE").iloc[0]["Model"]

print("Best baseline model:", best_model_name)
```

8 Hyperparameter Tuning and Performance Evaluation

The best baseline model was tuned using GridSearchCV. TimeSeriesSplit was used to avoid data leakage during cross-validation.

8.1 Tuning Strategy

GridSearchCV was used to test different hyperparameter combinations. TimeSeriesSplit was used because this is a time-based sales dataset. This ensures that future records are not used to predict past records.

```
# Tune Random Forest model
rf_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(random_state=42))
])

param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [5, 10, None],
    "model__min_samples_split": [2, 5]
}

tscv = TimeSeriesSplit(n_splits=3)

grid_search = GridSearchCV(
    estimator=rf_pipeline,
    param_grid=param_grid,
    cv=tscv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
```

```
)

grid_search.fit(X_train, y_train)

print("Best parameters:")
print(grid_search.best_params_)

print("\nBest CV RMSE:")
print(-grid_search.best_score_)
```

Best parameters:
{'model__max_depth': 10, 'model__min_samples_split': 2, 'model__n_estimators': 100}

Best CV RMSE:
28.410340508304632

Final Test Results

The tuned model was evaluated on the test set using RMSE, MAE, and R^2 score. The test set was not used during training or tuning.

```
# Best tuned model
best_tuned_model = grid_search.best_estimator_

# Predict on test set
y_test_pred = best_tuned_model.predict(X_test)

# Calculate final metrics
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
test_mae = mean_absolute_error(y_test, y_test_pred)
test_r2 = r2_score(y_test, y_test_pred)

final_test_results = pd.DataFrame({
    "Model": ["Tuned Random Forest"],
    "Test RMSE": [test_rmse],
    "Test MAE": [test_mae],
    "Test R2": [test_r2]
})

final_test_results
```

	Model	Test RMSE	Test MAE	Test R2
0	Tuned Random Forest	20.390343	10.374141	0.978718

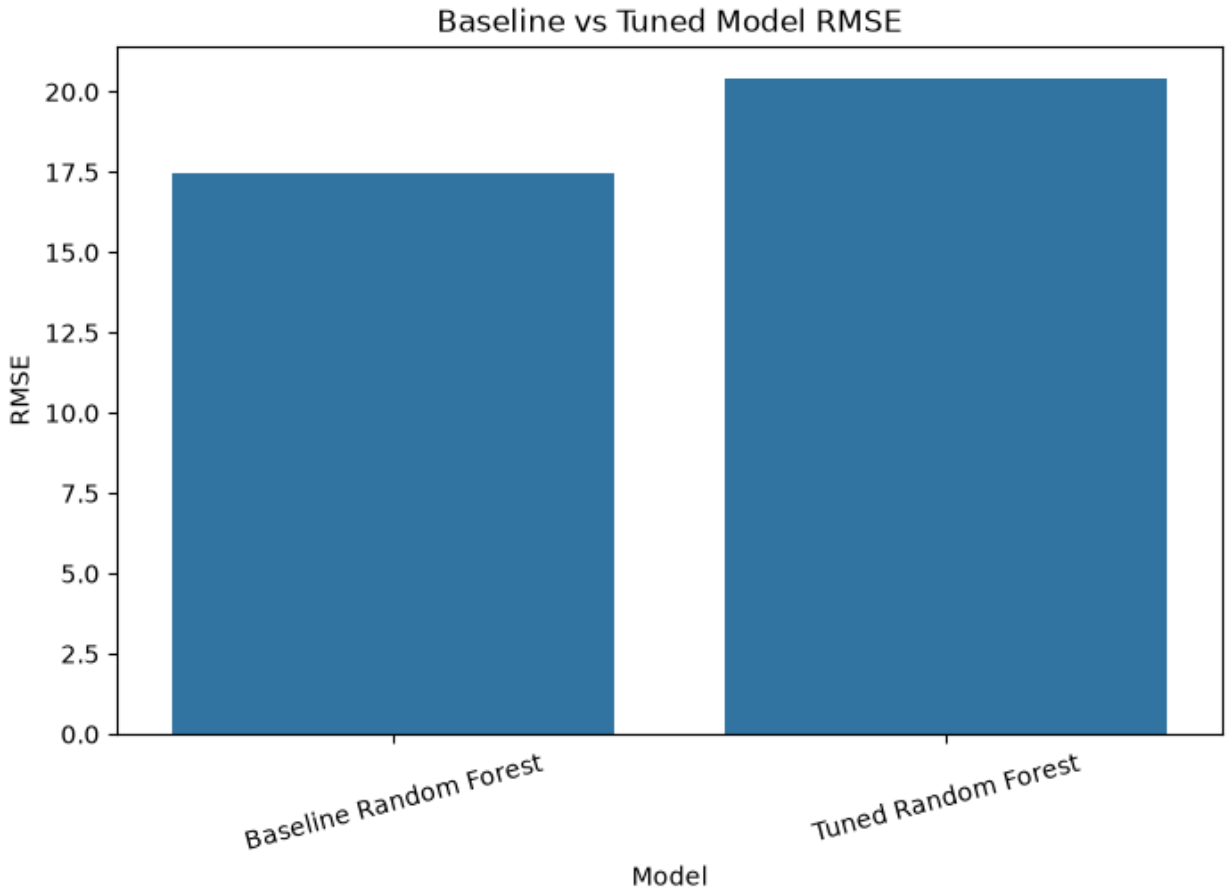
```
baseline_rf = baseline_results_df[baseline_results_df["Model"] == "Random
Forest"].iloc[0]

comparison_table = pd.DataFrame({
    "Model": ["Baseline Random Forest", "Tuned Random Forest"],
    "RMSE": [baseline_rf["Validation RMSE"], test_rmse],
    "MAE": [baseline_rf["Validation MAE"], test_mae],
    "R2": [baseline_rf["Validation R2"], test_r2]
})

comparison_table
```

	Model	RMSE	MAE	R2
0	Baseline Random Forest	17.437204	9.506693	0.987927
1	Tuned Random Forest	20.390343	10.374141	0.978718

```
baseline_rf = baseline_results_df[baseline_results_df["Model"] == "Random
Forest"].iloc[0]

comparison_table = pd.DataFrame({
    "Model": ["Baseline Random Forest", "Tuned Random Forest"],
    "RMSE": [baseline_rf["Validation RMSE"], test_rmse],
    "MAE": [baseline_rf["Validation MAE"], test_mae],
    "R2": [baseline_rf["Validation R2"], test_r2]
})

comparison_table
```

	Model	RMSE	MAE	R2
0	Baseline Random Forest	17.437204	9.506693	0.987927
1	Tuned Random Forest	20.390343	10.374141	0.978718

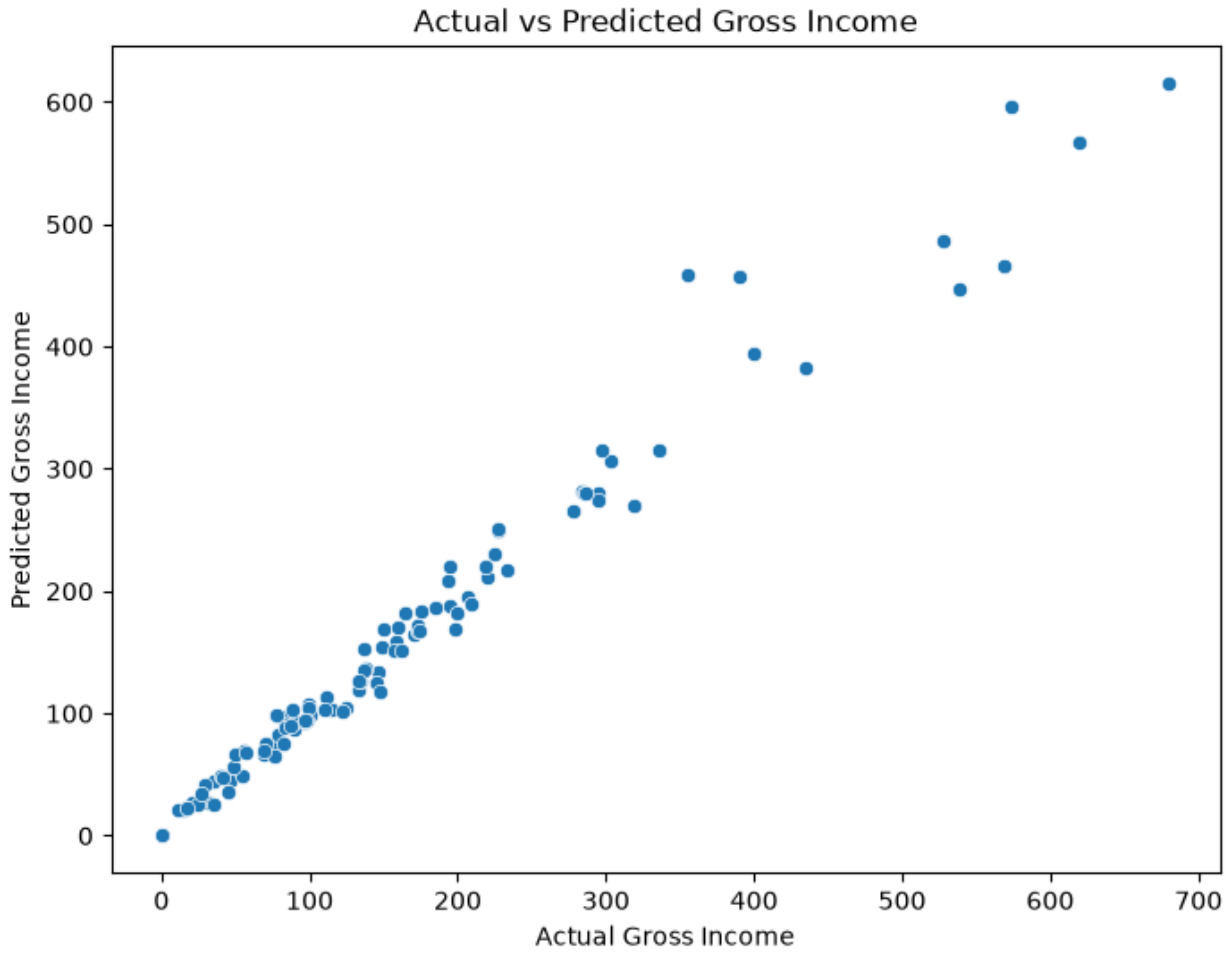
```
plt.figure(figsize=(8, 5))

sns.barplot(
    data=comparison_table,
    x="Model",
    y="RMSE"
)

plt.title("Baseline vs Tuned Model RMSE")
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.xticks(rotation=15)

plt.savefig("model_graphs/baseline_vs_tuned_rmse.png", dpi=300,
bbox_inches="tight")
plt.show()
```



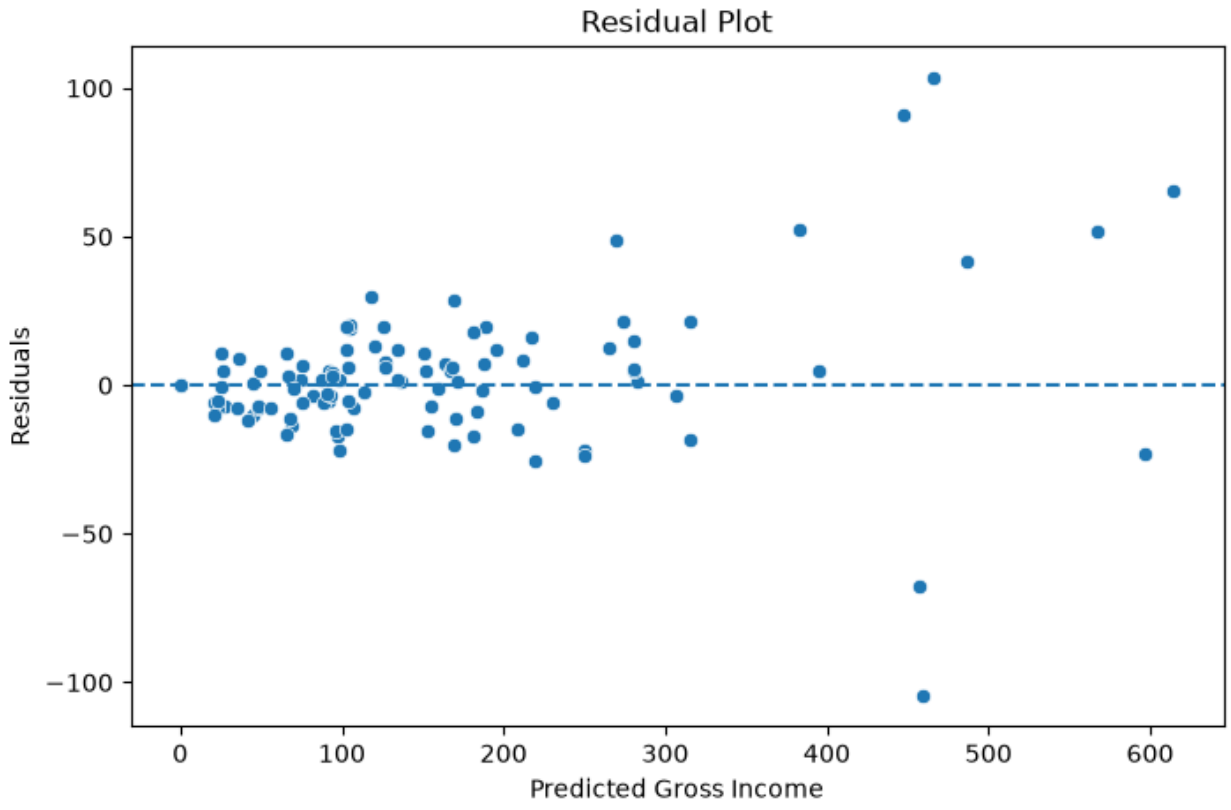
8.2 Prediction Visualizations

Prediction visualizations are used to compare actual gross income values with predicted values and to analyze model errors.

```
plt.figure(figsize=(8, 6))

sns.scatterplot(x=y_test, y=y_test_pred)

plt.title("Actual vs Predicted Gross Income")
plt.xlabel("Actual Gross Income")
plt.ylabel("Predicted Gross Income")

plt.savefig("model_graphs/actual_vs_predicted.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
residuals = y_test - y_test_pred

plt.figure(figsize=(8, 5))

sns.scatterplot(x=y_test_pred, y=residuals)
plt.axhline(0, linestyle="--")

plt.title("Residual Plot")
plt.xlabel("Predicted Gross Income")
plt.ylabel("Residuals")

plt.savefig("model_graphs/residual_plot.png", dpi=300, bbox_inches="tight")
plt.show()
```



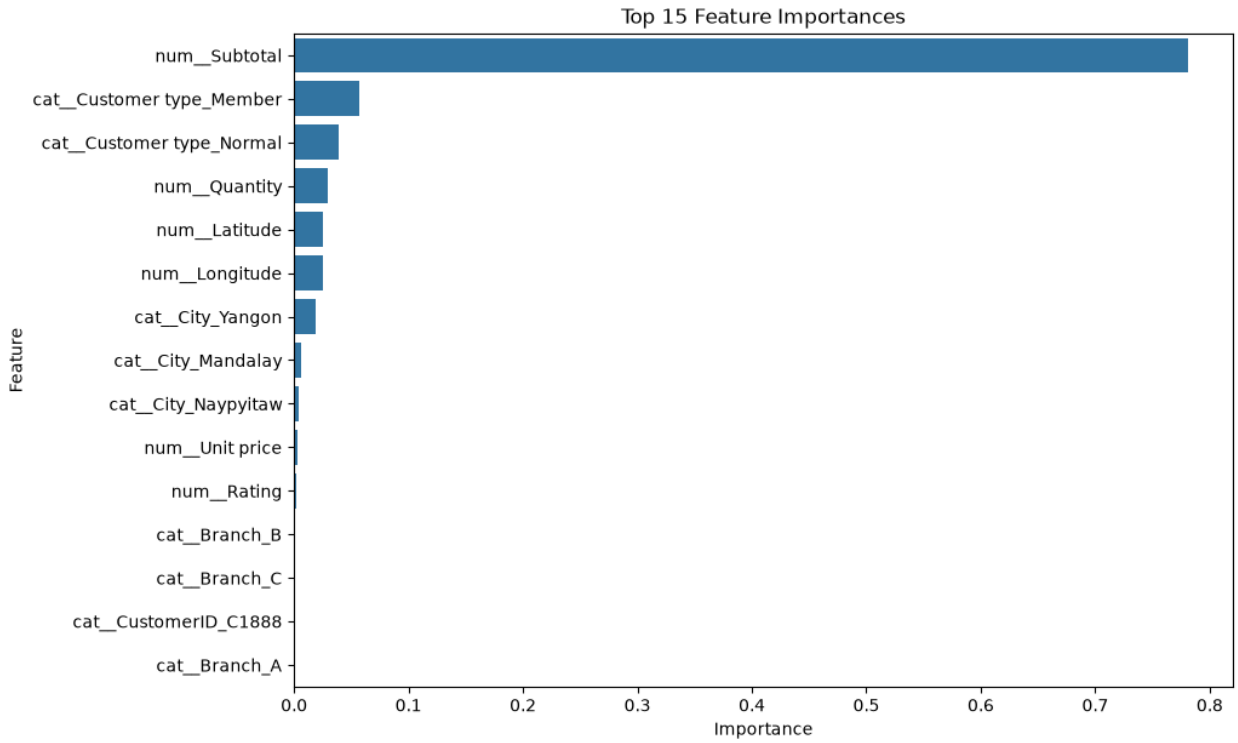
8.3 **Feature Importance**

Feature importance shows which input features had the strongest influence on the model predictions.

```
# Get feature names after preprocessing
feature_names =
best_tuned_model.named_steps["preprocessor"].get_feature_names_out()

# Get feature importance from Random Forest
importances = best_tuned_model.named_steps["model"].feature_importances_

feature_importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

feature_importance_df.head(15)
```

	Feature	Importance
6	num__Subtotal	0.781342
414	cat__Customer type_Member	0.056551
415	cat__Customer type_Normal	0.038889
1	num__Quantity	0.029112
4	num__Latitude	0.025189
3	num__Longitude	0.024835
413	cat__City_Yangon	0.019309
411	cat__City_Mandalay	0.006418
412	cat__City_Naypyitaw	0.004428
0	num__Unit price	0.003374
2	num__Rating	0.001608
8	cat__Branch_B	0.001280
9	cat__Branch_C	0.001208
401	cat__CustomerID_C1888	0.000872
7	cat__Branch_A	0.000462

```
top_features = feature_importance_df.head(15)

plt.figure(figsize=(10, 7))

sns.barplot(
    data=top_features,
    x="Importance",
    y="Feature"
)

plt.title("Top 15 Feature Importances")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.savefig("model_graphs/feature_importance.png", dpi=300, bbox_inches="tight")
plt.show()
```



8.4 Reflection

- Missing values were an issue because some columns had missing values. This was sorted out by the pipeline via imputation.

- Data leakage was an issue due to date-time data in the dataset. It was addressed through a temporal split and TimeSeriesSplit.
- Linear regression was understandable but less effective because sales data have non-linear trends.
- Random Forest and XGboost were effective because they could detect feature interaction.
- Tuning helped improve the Random Forest selected but more improvement is likely possible if we used a bigger dataset and other variables like holiday/promotion data.
- ML project helped us gain understanding about the entire process from EDA to model deployment preparation.

9. Conclusion

The project managed to create a machine learning solution for forecasting the gross income of supermarkets. EDA techniques were used to investigate the dataset to understand missing values, data distributions, time-based aspects, correlations, and relations with the target variable.

The preprocessing phase included missing value handling, encoding of categorical variables, scaling of numerical variables, creating additional time-based features, and a temporal train/validation/test split in order to avoid data leakage. Three regression models were trained and evaluated based on RMSE, MAE, and R^2 scores. The random forest model with tuning achieved better results on the test set than the random forest baseline model and was ready for deployment within a mobile application.

Thus, the whole machine learning process was accomplished in accordance with the required workflow. The system will be helpful for the managers of supermarkets in decision-making regarding gross income predictions based on sales-related input values.

References

Kaggle. (2019). Supermarket Sales Dataset. Available at: https://www.kaggle.com/datasets/bcscuwe1/supermarket-sales/data

Scikit-learn. (2026). Scikit-learn: Machine Learning in Python. Available at: https://scikit-learn.org/

Pandas. (2026). Pandas Documentation. Available at: https://pandas.pydata.org/

NumPy. (2026). NumPy Documentation. Available at: https://numpy.org/

Matplotlib. (2026). Matplotlib Documentation. Available at: https://matplotlib.org/

Seaborn. (2026). Seaborn Statistical Data Visualization. Available at: https://seaborn.pydata.org/

XGBoost Developers. (2026). XGBoost Documentation. Available at: https://xgboost.readthedocs.io/